

Figure 1: Dev-Ops architecture

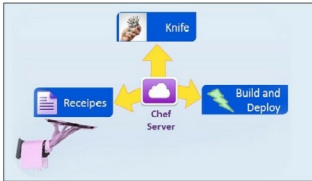


Figure 2: Components in Chef Server

test and to promote code builds into various server zones. This is also technically termed as 'turning infrastructure into code'.

Later, we plan on writing a series of articles on Dev-Ops. For now, let's start with an introduction to Chef, its journey so far, and then install and configure Enterprise Chef on RHEL 7, which will help you to create a Chef platform to work further.

The evolution of Chef

Chef was first released in January 2009 in Linux based continuous deployment (CD); it stabilised around 2010 in Linux as well as Windows. Its original name was Marionette, when Adam Jacob created it as a pilot for automating the build/deployment process for one of his clients. The potential of this tool was realised when it began handling automation efficiently in Amazon work servers. That's when it was converted to a product and named Chef. It is based on scripts called recipes, which run to push components for deployment and do other build/deployment activities. This set of recipes for a unique deployment environment is called a cookbook. The name Chef, therefore, is so apt and the tool used by Chef for such deployment is also called a knife. Chef and its supporting tools are written using the Ruby language.

Chef incorporates in a unified convenient place, all the development, build, deployment and monitoring tools and services needed to move a software product from initialisation and set-up, through development, on to testing and, finally, release.

Chef enables organisations to quickly deploy, manage and scale existing and new enterprise-grade applications on any infrastructure, with no changes to the code or architecture. It helps to maximise application onboarding and automation by externally orchestrating the application deployment and runtime. The Chef cookbook based approach treats

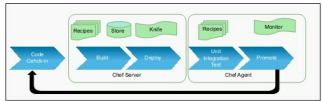


Figure 3: Chef operations in the server and agent

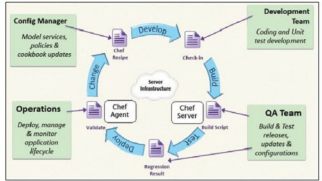


Figure 4: Chef tools

infrastructure as code, enabling you to describe deployment and post-deployment steps for any application through an external blueprint called a recipe, which you can then take from one infrastructure to another, unchanged.

Chef is highly useful in cloud based deployment as well, and simplifies cloud migration by provisioning the compute resources you need, on demand, on the cloud of your choice (public cloud, private cloud, or even hybrid cloud), using its deployment tool to push to the cloud. You can define scaling rules based on any custom metric—and Chef will scale out or scale in as needed. Technically, Chef enables you to install your application with a single shell or REST (Representation and State Transfer) API, with no code changes, and also takes your infrastructure monitoring capabilities to the next level. You can easily monitor your application services for availability and performance, define any custom metrics you want to use, and ensure business continuity at all times.

Chef's components

Chef Server or Enterprise Chef can be installed on the Windows platform, Mac OS X or on the Linux platform. We have two flavours of Chef available in Linux—the Red Hat Enterprise version and the Ubuntu version.

The Chef client is available in UNIX flavours like FreeBSD, Linux, AIX and Solaris, in addition to Windows and Mac OS. Hence, we can set up Chef nodes in all these machines and integrate with a primary hosted Chef server.

Chef has two flavours of installation, namely, Hosted Chef and Standalone or Enterprise Chef. Let us discuss them briefly.

Hosted Chef runs on a preconfigured cloud environment, where the user has to reserve a space for deployment activities and then upload code for automating build, deployment, testing and to promote the build. All required